Naming convention in java ?

---

* While delving the java code, It is **strongly recommended** to follow the naming convention.
* We have following 2 advantages to follow the naming convention

1) The reliability of the code will be increased.
2) Industry Standard Code.

Naming convention for java classes :

---

* While writing the java classes we should follow, Pascal naming convention which describes each word first letter must be capital and should not contain any space. A java class is represented by noun.

Example : **ThisExampleOfClass**

String
System
Integer
BufferReader
ArrayIndexOutOfBoundsException

Naming convention for java methods :

While writing a method in java we should follow camel case naming convention, According to this naming convention first word will be in small and 2nd word onwards, each word first character must be capital. So java a method represents verb.

Example :
thisIsExampleOfMethod()
add()
readLine()
parseInt()
charAt();
toUpperCase()

Naming convention for a Variable :

While writing a variable we should follow camel case naming convention but unlike method variable does not have () symbol.

Example :

---

rollNumber
studentName
customerBill
studentName
playerName

Naming convention for writing final and static (constants) variables :

---

* While writing final and static variable, We should follow snake case naming convention. According to this, Each character must be capital and each word must be separated by _ symbol.

Example : MIN_VALUE
MAX_VALUE
SIZE

Naming convention for writing a package in java :

---

* Package name must be in lower case only. Generally we should write reverse of company name.

Example : com.ksrinivas.net
com.ntpro.command_line_argument

Tokens in java :

Token is the smallest unit of the program which is identified by the compiler.

Without token we can't complete statement or expression in java.

Token is divided into 5 types in java

1) Keyword
2) Identifiers
3) Literal
4) Punctuators (Separators)
5) Operator

Keywords :

---

* It is a predefined word whose meaning is already defined by java compiler.
* A keyword we cannot use as an identifier in java.
* All the keywords must be in lower case only.
* true, false and null look like keywords but actually they are literals.
* As of now, We have 67+ keywords in java.

Identifiers :

A name in java program by default considered as identifiers.

Assigned to variable, method, classes to uniquely identify them.

We can't use keyword as an identifier.

Ex:-

---

class Fan
{
int coil ;

void switchOn()
{
}
}

Here Fan(Name of the class), coil (Name of the variable) and switchOn(Name of the Method) are identifiers.

Rules for defining an identifier :

---

1) Can consist of uppercase(A-Z), lowercase(a-z), digits(0-9), $ sign, and underscore (_)
2) Begins with letter, $, and _
3) It is case sensitive
4) Cannot be a keyword
5) No limitation of length

Literal :

---

* Any constant value which we can assign to a variable is called Literals.

In Java, As of now we have 5 types of Literals which are as follows :

1) Integral Literal
2) Floating Point Literal
3) Boolean Literal
4) Character Literal
5) String Literal

Note : Along with these we have null and class constant literal.

Integral Literal :

---

* If any numeric literal **does not contain** decimal OR fraction then it is called Integeral Literal.
Example : 12, 90, 56

* In integral literal we have 4 data types

a) byte (8 bits)
b) short (16 bits)
c) int (32 bits)
d) long (64 bits)

* An integral literal we can represent 4 ways in java

a) Decimal Literal (Base 10)
b) Octal Literal (Base 8)
c) Hexadecimal Literal (Base 16)
d) Binary Literal (Base 2) [It is available from JDK 1.7V onwards]

a) Decimal Literal :

---

* Any integral literal by default represents decimal Literal. The base is 10 so, It will accept 10 digits i.e. 0 to 9.

Example :

int x = 90;
int y = 89;
int z = 78;

b) Octal Literal :

If an integral literal starts with **0 (zero)** then it will become Octal literal. The base is 8 so It will accept digits i.e 0 to 7 only.

Example :

int x = 015; //Valid
int y = 089; //Invalid
int z = 018; //Invalid [digit 8 is out of the range]

Programs :

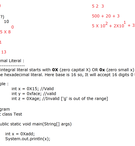
---

public class Test
{
public static void main(String[] args)
{
int x = 015;
System.out.println(x); //13
}
}

How the output of above program is 13

---

* As a Java developer, We can represent an integral literal in 4 different forms i.e Decimal, Octal, Hexadecimal and Binary but to generate the output JVM will convert these number System into decimal because decimal is default number system.



015 -> Here the number 0 describes that it is octal literal

5 2 3

500 = 20 x 1

5 x 10^2 + 2x10^1 + 3x10^0

8 + 5 x 1

8 + 5 = 13

Hexadecimal Literal :

---

* If an integral literal starts with **0X** (zero capital X) OR **0x** (zero small x) then it will become hexadecimal literal. Here base is 16 so, It will accept 16 digits 0 to 9 and A-F.

Example :

int x = 0X15; //Valid
int y = 0xface; //valid
int z = 0xage; //Invalid ['g' is out of the range]

//Program
public class Test
{
public static void main(String[] args)
{
int x = 0XABE;
System.out.println(x);
}
}

0xabE -> Here 0X represents, It is hexadecimal number

a = 10
b = 11
c = 12
d = 13
e = 14
f = 15

A X 16^2 + B X 16^1 + E X 16^0

10 X 256 + 13 X 16 + 13 X 1

2560 + 208 + 13 = 2781